2025 8th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE) | 979-8-3315-1091-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/AEMCSE65292.2025.11042839

A Survey of Low-Rank Adaptation Techniques

Zeqin Feng
Institute of Systems Engineering
Academy of Military Science
Beijing, China
13145647181@163.com

Yongqiang Xie
Institute of Systems Engineering
Academy of Military Science
Beijing, China
yqxie_2024@163.com

Jiawei Yang
Institute of Systems Engineering
Academy of Military Science
Beijing, China
19972880263@163.com

Wenjun Hou
Unit 91977
Beijing, China
15810687609@163.com

Zhongbo Li*
Institute of Systems Engineering
Academy of Military Science
Beijing, China
*lzb05296@163.com

***Abstract*—The advent of the Transformer architecture has spurred the development of Large Language Models (LLMs), yet fine-tuning these models for downstream tasks poses challenges due to high computational costs. Low-Rank Adaptation(LoRA) has emerged as a leading parameter-efficient fine-tuning (PEFT) method，LoRA is based on the Low Intrinsic Dimensionality Hypothesis, reducing trainable parameters through low-rank matrices decomposition while maintaining or enhancing performance. This paper comprehensively reviews LoRA, covering its low-rank adaptation concept, technical optimizations, architectural innovations, existing challenges and future prospective.**

Keywords-Low-Rank Adaptation, Parameter-Efficient Fine-Tuning, Large Language Models

I. INTRODUCTION

The proposal of Transformer[1] architecture in 2017 elevated natural language processing technology to a new dimension, and the innovation of this technical architecture laid the foundation for the birth of iconic Large Language Models (LLMs) such as GPT-4[2] and LLaMA[3]. In the past few years, the parameter sizes of pre-trained language models (PLMs) have rapidly increased to reach billions and even trillions for some large models, resulting in outstanding emergence and generalization capabilities, which have become an indispensable part of modern artificial intelligence systems. However, the introduction of LLMs such as Deepseek-V3[4] and the open source of its parameters have announced that the performance of industry-leading models can still be achieved with limited arithmetic power and parameter sizes by virtue of technological innovation, which provides a new way of thinking to improve the performance of AI. Nevertheless, LLMs still need to be fine-tuned to achieve the corresponding performance effect when researchers face the actual downstream tasks due to their own knowledge boundaries.

During the actual fine-tuning process for specific downstream tasks, the computational overhead of full fine-tuning for LLMs is prohibitively high. For instance, full fine-tuning of the DeepSeek-7B[5] model requires 60 GB of GPU memory, necessitating two professional-grade A100 (40 GB) GPUs for training—a resource demand that is particularly challenging for independent researchers with limited funding. To address this challenge, researchers have developed various parameter-efficient fine-tuning (PEFT) methods. Categorized by their operational characteristics during fine-tuning, mainstream PEFT[6] approaches primarily include adapter tuning[7], prompt tuning[8], and Low-Rank Adaptation[9]. Notably, the LoRA framework and its derivatives (e.g., QLoRA[10], AdaLoRA[11], Flora[12]) have emerged as predominant solutions in the PEFT paradigm, owing to their architectural simplicity, computational efficiency, and versatile applicability across diverse model architectures and research domains.

The LoRA methodology operates by freezing the original model parameters during PEFT, focusing on updating low-rank matrices injected into the Transformer layers of LLMs. This approach reduces trainable parameters to one-thousandth of those required for full fine-tuning—for instance, fine-tuning a model like GPT-3 175B[13] demands merely 0.01% of parameters while reducing GPU memory consumption to one-third of full fine-tuning requirements. Remarkably, it achieves performance comparable to or even surpassing that of full fine-tuning. A distinctive advantage of LoRA lies in its post-training operational efficiency: the optimized low-rank matrices can be seamlessly merged with the original weight matrices through linear algebra operations, incurring no additional computational overhead. Furthermore, when adapting to diverse downstream tasks, only task-specific low-rank modules need activation, thereby slashing storage overhead and computational switching costs by orders of magnitude.

In this review, we give a comprehensive review of the current research progress as in Fig. 1 of LoRA methods, mainly including concept of low-rank adaptation, technical optimization, architectural innovation, and conclusion.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

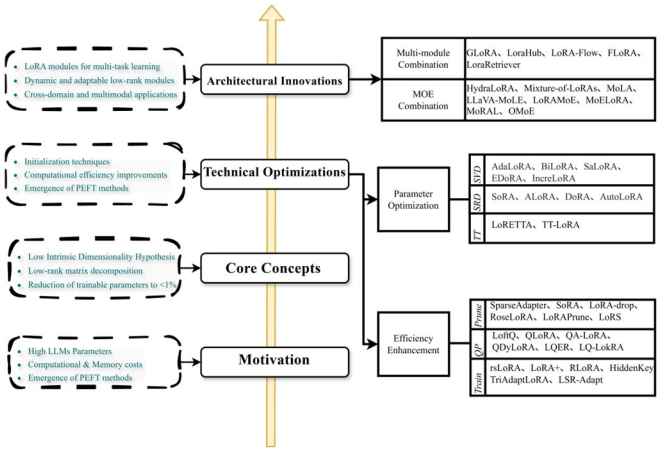


Fig. 1 Structure of LoRA survey.

II. Concept of Low-Rank Adaptation

The Low Intrinsic Dimensionality Hypothesis[14] posits that most pre-trained models inherently exhibit a low-dimensional reparameterization space, where adapting to diverse task requirements can be achieved by adjusting only a small subset of parameters. Building on this theoretical foundation, LoRA emerges as a parameter-efficient fine-tuning methodology. Its core innovation lies in constraining weight updates via low-rank matrix decomposition, thereby drastically reducing trainable parameters (typically to <1% of full fine-tuning) while preserving—and often enhancing—task-specific performance [15]. This framework enables the creation of domain-specific LoRA modules tailored to distinct application scenarios, facilitating efficient downstream task switching through shared base model parameters with task-adaptive low-rank components.

In this section, we elucidate the technical principles and operational advantages underpinning LoRA.

A. Basic Concepts of Low-Rank Adaptation

The LoRA[9] method freezes the weight matrix $W_0 \in R^{k\times d}$ of the pre-trained model without receiving gradient updates while adapting the pre-trained model to the downstream task, while decomposing the weight update matrix ΔW into two low-rank matrices $B \in R^{d\times r}$ and $A \in R^{r\times k}$, containing the trainable parameters, where r is the rank of the matrices, and the rank $r \ll min(d,k)$.

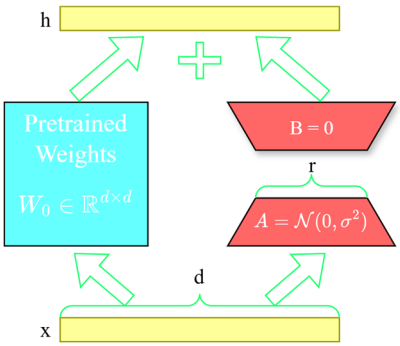


Fig. 2 Illustration of LoRA adapters[9].

As shown in the Fig. 2, the weight update matrix ΔW can be expressed as in (1), and a specific initialization method is applied to the low-rank matrices, which is initialized with a randomized Gaussian distribution for the low-rank matrix A and zero initialization for the matrix B, This ensures that ΔW is actually a zero matrix at the beginning of the fine-tuning.

$$\Delta W=BA \tag{1}$$

This principle makes ΔW in the fine-tuning process low-rank, reduces the original d×k parameters that need to be updated to (d+k)×r, parameters, reduces the number of parameters involved in training, and greatly reduces the computational and storage costs.

During the fine-tuning process, the low-rank matrices A and B serve as the sole trainable parameters, dedicated to learning task-specific adaptations. In the forward propagation phase, the original weight matrix W_0 operates in parallel with the low-rank update matrix ΔW. For an input vector x, the output is computed through dual pathways:

The baseline projection: W_0x

The low-rank adaptation term as in (2).

$$\Delta Wx=(\alpha/r)BAx \tag{2}$$

The final hidden representation is formulated as in (3).

$$h=W_0x+\Delta Wx=W_0x+(\alpha/r)BAx \tag{3}$$

Here, the scaling factor α (typically initialized as a constant tied to the rank r) controls the adaptation strength. This design inherently minimizes hyperparameter tuning demands, as α/r provides stable gradient scaling without requiring extensive empirical adjustments.

B. Advantages of LoRA

Compared to full fine-tuning of large language models, LoRA has the following advantages:

1) Small-scale parameters, high training efficiency.

Utilizing the principle of low-rank decomposition reduces the trainable parameters by several orders of magnitude, accelerating the convergence of the training process while greatly reducing the consumption of computational resources, especially in resource-constrained environments or scenarios requiring multiple fine-tuning, which gives the LoRA method a unique advantage.

2) No inference delay, flexible model adaptation.

The parameter update matrix is explicitly incorporated into the frozen weight matrix without increasing the time loss in the inference process, maintaining the same inference speed as the original model, with the ability to interchange multitasking learning adapters under the same model architecture, and minimizing storage requirements to avoid additional computational overhead.

3) Mitigating catastrophic forgetting and dynamically deploying task extensions.

The freezing of the original weight matrix ensures that the model learns task-specific patterns only by merging lightweight low-rank matrices when adapting to new tasks, ensuring that the underlying language comprehension and reasoning capabilities are not overwritten, and that a single LoRA adapter (about 10MB in size for the 7B model) can be saved independently to

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

support dynamic loading at runtime. When switching between translation, summarization, and Q&A tasks, only the adapter files need to be replaced, eliminating the need to maintain multiple copies of the full model.

III. Technical optimization

This section will summarize the basic problems and important breakthroughs of lora technology from two key aspects: Parameter optimization and Training process improvement. Parameter optimization includes two dimensions: Parameter decomposition and Efficiency enhancemen.

A. Parameter Optimization

The rapid development of LLMs has led to their enormous parameter scales. For models with parameter scales like Deepseek-V2[16], even when using the highly parameter-efficient LoRA method to fine-tune just 5% of the model with the float16 data type, it inevitably requires 640GB of GPU memory. To address this challenge, we summarize the current technical research from three aspects: parameter decomposition, pruning and quantization.

In the LoRA method during PEFT, the incremental update budget is typically allocated uniformly, overlooking the fact that weight matrices in different modules and layers have varying degrees of importance. Inspired by the concept of exploring effective rank, researchers have focused on reducing the number of trainable parameters by decomposing fine-tuning matrices into smaller scales. This allows for a more microscopic grasp of the training process while improving both task performance and parameter efficiency. For the decomposition of update parameter matrices, there are currently three main approaches: methods based on singular-value decomposition (SVD), methods based on single-rank decomposition (SRD), and methods based on tensor-train decomposition.

1) Methods based on Singular-Value-Decomposition.

Due to the SVD decomposition inspired decomposition of the parameter matrix decomposition into PΛQ, where P and Q are orthogonal and Λ is a non-negative diagonal matrix. By controlling the elements in Λ, the filtered diagonal matrix can be controlled to assign rank to the LoRA module, enabling dynamic control of the matrix rank and adaptive parameter assignment. The model equation is (4)

$$h=W_0x+\Delta Wx=W_0x+(P\Lambda Q)x \tag{4}$$

The SVD decomposition provides a unified framework for the rank adaptive control of LoRA modules, while different methods gradually address the balance between efficiency and performance by dynamically adjusting the balance between the strategy and optimization objectives. AdaLoRA[11] parametrically fine-tunes the weights "ΔW" through singular value decomposition and uses regularization to ensure the orthogonality of P and Q to approximate the SVD decomposition. And further dynamically prunes the minor singular values in Λ based on the importance score to improve the parametric efficiency.

To solve the overfitting problem, BiLoRA[17] introduces a two-layer optimization framework to separate the training of singular vectors and singular values to different data subsets. This separation strategy effectively mitigates overfitting while improving the generalization ability of the model. Meanwhile, SaLoRA[18] is further optimized by introducing a fixed safety module and task-specific initialization to ensure that the safe alignment of the model is preserved during fine-tuning, and by employing orthogonal regularization to filter the singular values through the L_0 paradigm.

EDoRA[19], on the other hand, achieves a significant reduction in the number of parameters by decomposing the pre-training weights into the magnitude and directional components, and by freezing the low-rank matrices, introducing a small trainable matrix significant reduction in the number of parameters while maintaining the learning capability. IncreLoRA[20] proposes a single-rank starting strategy that automatically increases the rank based on heuristic scoring and relaxes the non-negative constraints on Λ to improve flexibility. This approach avoids the limitation of preset rank budget and further breaks the efficiency bottleneck of traditional methods. A comparison of specific features is shown in TABLE I.

TABLE I. Comparison of SVD method.

Variant	Initialization Strategy	Screening Basis	Constraints	Efficiency Characteristics
AdaLoRA	High-rank	Importance Score	$\Lambda \geq 0$, Orthogonality	Time-consuming Iterative Rank Reduction
BiLoRA	High-rank	Validation Loss within Optimization Framework	Orthogonality	Dual-layer Optimization to Prevent Overfitting
SaLoRA	High-rank	L_0 Norm	$\Lambda \geq 0$, Orthogonality	Sparse Constraint Limited
EDoRA	Low-rank Frozen Matrix	SVD Initialization	Orthogonality	Freeze Matrix to Reduce Parameters
IncreLoRA	Single-rank	Heuristic growth score	$\Lambda \in \mathbb{R}$, Orthogonality	Budget-free self-adaptation

2) Methods based on Single-Rank decomposition.

Orthogonal regularization helps to maintain parameter independence, but introduces a significant computational overhead reducing the efficiency. To address this limitation, recent advances have developed dual decomposition strategies through different approaches. The first studies focus on updating the matrix decomposition by omitting the SVD orthogonality requirement. These methods directly decompose the incremental update matrix ΔW into single-rank components through an adaptive pruning mechanism.

SoRA[21] presents a PEFT method for large language models by dynamically adjusting the intrinsic rank during adaptation through a sparse gating unit optimized with a proximal gradient method. This approach initializes with a higher rank but efficiently tames the temporarily increased parameters via sparse updates. SoRA also introduces a sparsifying scheduler to explore the relationship between the number of non-zero parameters and the model's memorization and generalization capabilities. Experiments indicate that SoRA can outperform other baselines even with 70% retained parameters and 70% training time.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

ALoRA[22] proposes a framework that allocates LoRA ranks dynamically during the fine-tuning process. It introduces AB-LoRA to estimate the importance score of each LoRA rank and then prunes abundant ranks while reallocating the pruned budgets to important Transformer modules requiring higher ranks. The method demonstrates superior performance over recent baselines with comparable tunable parameters.

DoRA[23] decomposes pre-trained weights into magnitude and direction components for fine-tuning, employing LoRA specifically for directional updates to minimize trainable parameters. The weight decomposition analysis reveals distinct patterns between LoRA and full fine-tuning. DoRA consistently outperforms LoRA across various tasks without additional inference overhead.

AutoLoRA[24] is a meta-learning-based framework that automatically determines the optimal rank for each LoRA layer. It associates selection variables with each rank-1 matrix in a low-rank update matrix to control the rank and employs meta learning to optimize these variables. The optimal rank is determined by thresholding the selection variables, followed by retraining the update matrices. Experiments show AutoLoRA's effectiveness in finding optimal ranks for different layers, outperforming baselines across various NLP tasks. A comparison of specific features is shown in TABLE I.

TABLE II. COMPARISON OF SRD METHOD

Variant	Decomposition Components	Sparsity	Dynamic Rank	Mechanism	Application Scope
SoRA	$\Delta W=B(g\odot W_d)$	Yes	Yes	Dynamically adjusts rank during training by pruning zeroed-out ranks	NLP tasks
ALoRA	$\Delta W=B(g\odot W_d)$	No	Yes	Allocates ranks dynamically across different Transformer modules	NLP tasks
DoRA	Magnitude & Direction	No	No	Decomposes weights into magnitude and direction, updates direction with LoRA	NLP and Vision-Language tasks
AutoLoRA	$\Delta W=\sum a_j \Delta j$	No	Yes	Automatically determines layer-specific ranks using meta learning	NLP tasks (NLU, NLG, sequence labeling)

3) Methods based on Tensor-Train Decomposition

The Tensor-Train(TT) decomposition[25], first introduced by Oseledets in 2011, offers an efficient approach to representing high-dimensional tensors by breaking them into a sequence of smaller three-dimensional tensor cores. This method avoids the curse of dimensionality by significantly reducing computational complexity and memory usage while preserving the essential structure of the data. The TT format has been successfully applied to solve high-dimensional problems in fields such as quantum mechanics, stochastic partial differential equations, and financial modeling.

LoRETTA[26], with its two methods—LoRETTAadp and LoRETTArep—uses tensor-train decomposition to reduce trainable parameters. LoRETTAadp employs tensorized adapters for a lightweight yet high-performance approach, while LoRETTArep focuses on reparameterizing weights with small tensor factors.

Recent research has explored the application of TT-decomposition to PEFT of LLMs. TT-LoRA[27] leverages TT-decomposition to compress weight update matrices in LLMs, achieving substantial model compression without compromising performance. By decomposing weight matrices into smaller tensor cores, TT-LoRA eliminates the need for traditional LoRA based structures or adapters, thereby reducing inference latency and computational overhead. Experimental results demonstrate that TT-LoRA achieves impressive compression ratios while maintaining or even improving performance compared to full fine-tuning and other PEFT methods. This approach highlights the potential of TT-decomposition as a key technique for enabling efficient deployment of large scale LLMs on resource constrained platforms.

B. Efficiency Enhancement

Although LoRA achieves improved parameter efficiency, the method still requires a large number of trainable parameters. This section analyzes existing solutions from the perspectives of parameter pruning, quantization and training process improvement.

1) Parameter Pruning

LoRA optimization methods enhance efficiency by dynamically eliminating redundant parameters during training and inference. Through systematic importance evaluation, these techniques selectively prune adaptation components either in isolation or through coordinated refinement with base model parameters, maintaining performance while reducing computational load.

SparseAdapter[28] employs network pruning at initialization to eliminate redundant parameters, achieving comparable or better performance than standard adapters even at 80% sparsity. This method effectively reduces the parameter budget while maintaining model capacity. SoRA[21] extends LoRA by incorporating a gate unit optimized with a proximal gradient method, enabling dynamic adjustment of the intrinsic rank during training. This allows for efficient parameter updates and improved representation power, particularly excelling in tasks requiring adaptive rank selection.

LoRA-drop[29] evaluates parameter importance based on the output of LoRA, rather than just parameter features, leading to significant reductions in trainable parameters while preserving performance. This method identifies and retains critical parameters, making it highly effective for parameter-efficient fine-tuning. RoseLoRA[30] introduces row and column-wise sparsity constraints on the product of low-rank matrices, ensuring efficient and precise model updates. This approach is particularly advantageous for knowledge editing tasks, where selective parameter updates are essential.

LoRAPrune[31] proposes a memory-efficient pruning criterion that leverages LoRA gradients to estimate the

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

importance of pre-trained weights. This method facilitates structured pruning without the need to compute gradients of pre-trained weights, achieving high compression rates with minimal performance loss. LoRS[32] introduces weight recompute and computational graph rearrangement strategies to reduce memory and computation overhead, making it suitable for efficient fine-tuning of sparse LLMs. TABLE I. summarizes five pruning techniques for LoRA, highlighting their pruning approaches, limitations, and typical use cases. Each method offers unique strategies to enhance parameter efficiency and model performance through pruning.

TABLE III. COMPARISON OF PRUNING TECHNIQUES FOR LORA METHODS.

Variant	Innovation	Deficiency	Applicability
SparseAdapter	Pruning at initialization; effective at 80% sparsity	Limited to traditional methods	General NLP tasks
SoRA	Dynamic rank adjustment with gate unit	Potential computational overhead	Tasks needing adaptive rank selection
LoRA-drop	Importance evaluation via LoRA output	Coarser granularity	Significant parameter reduction with performance maintenance
RoseLoRA	Row and column-wise sparsity for precise updates	Requires hyperparameter tuning	Knowledge editing and precise parameter updates
LoRAPrune	Memory-efficient pruning using LoRA gradients	Requires fine-tuning	High compression in LLMs
LoRS	Weight recompute and computational graph rearrangement	Increased computation due to weight recompute	Efficient fine-tuning of sparse LLMs

TABLE IV. COMPARISON OF KEY FEATURES AND TECHNIQUES IN LORA QUANTIZATION VARIANTS.

Variant	Timing	Precision	Technique	Optimization	Memory Focus	Dequant
LoftQ	Pre-FT	mixed	Uniform & NormalFloat	Joint	FT	Partial
QLoRA	Pre-FT	4-bit	NormalFloat	Separate	FT&Inference	Partial
QA-LoRA	Pre&During FT	2, 3, 4-bit	Group-wise	Joint	FT&Inference	None
QDyLoRA	During FT	mixed	Rank sampling	Q-error rank min	Dynamic LoRA	None
LQER	Post-FT	mixed	Group-wise & adaptive	SVD-based LR	Inference	None
LQ-LokRA	Pre-FT	mixed	ILP & data-aware	Joint	FT&Inference	Partial

2) Parameter Quantization

Quantization plays an important role in optimizing the LoRA method, greatly reducing storage and computational requirements through low-precision representation. Early methods focused on uniform quantization, while recent advances have explored non-uniform and mixed-precision techniques to better align with the statistical properties of model weights.

QLoRA[10] introduced a novel 4-bit NormalFloat (NF4) quantization method, specifically designed for the Gaussian-like distribution of weights in pretrained models. This approach dynamically allocates more precision to smaller weights, which are prevalent in these models, thereby enhancing the representation of critical parameters and reducing memory usage. LoftQ[33] proposed an alternating optimization framework that integrates low-rank approximation with quantization.

By iteratively applying quantization and Singular Value Decomposition (SVD), LoftQ effectively minimizes quantization errors, ensuring that the quantized weights closely match the original high-precision weights. QA-LoRA[34] advanced the field by implementing group-wise quantization, allowing dynamic adjustment of precision during training. This method balances the interaction between low-rank updates and quantized weights, optimizing both memory efficiency and adaptation. LQER[35] focused on post-finetuning quantization, employing a low-rank SVD-based decomposition to minimize errors introduced during quantization. This method ensures that quantized weights are optimized for inference, maintaining high accuracy with reduced precision. QDyLoRA[36] introduced dynamic low-rank adaptation, enabling efficient fine-tuning across a spectrum of ranks within a single training cycle. This approach eliminates the need for multiple training rounds to find the optimal rank, streamlining the adaptation process. TABLE IV. provides a detailed summary of quantitative methods

3) Training Process Improvement

Although LoRA has demonstrated unique advantages in model fine-tuning scenarios with low parameter occupancy, continuous optimization of the training strategy is still required to unleash its full potential. By integrating recent research results, it is found that advances in learning rate and strategy optimization together constitute the current technological frontier in training efficiency optimization.

The evolution of LoRA techniques has seen significant advancements in parameter-efficient fine-tuning for large language models. Initially, rsLoRA[37] introduced a rank-stabilized scaling factor to address gradient collapse in high-rank adapters, enabling better performance with increased computational resources. Shortly after, LoRA+[38] optimized learning rates for adapter matrices A and B, demonstrating improved efficiency and speed in fine-tuning. RLoRA[39] further reduced trainable parameters using Randomized Walsh-Hadamard Transform, while incorporating PAC-Bayes regularization to enhance generalization.

Concurrently, HiddenKey[40] emerged as a unified dropout method tailored for LoRA, mitigating overfitting through strategic dropout mechanisms. TriAdaptLoRA[41] dynamically allocated trainable parameters using triangular splits and adaptive rank growth, inspired by synaptic plasticity in neuroscience. Finally, LSR-Adapt[42] leveraged matrix Low Separation Rank (LSR) representations to achieve ultra-efficient parameter tuning with fewer trainable parameters than conventional low-rank methods. Each method brought unique

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

innovations but also faced challenges such as computational overhead, sensitivity to hyperparameters, and the need for further optimization in dynamic threshold settings. TABLE I. provides a detailed summary of learning rate and optimization strategies.

TABLE V. COMPARISON OF DIFFERENT LORA VARIANTS IN TERMS OF LEARNING RATE ADJUSTMENT, OPTIMIZATION STRATEGIES.

Variant	Learning Rate	Optimization	Innovation Points	Drawbacks
rsLoRA	Scaling factor: $\gamma_r \in \Theta_r(1/\sqrt{r})$	Stabilizes gradients	Stabilizes high-rank learning	Limited to rank adjustments
LoRA+	$\eta_B \in \lambda\eta_{BA}$	Enhanced feature learning	Faster training, better performance	Tuning complexity, initialization sensitivity
RLoRA	Learnable scaling vector	PAC-Bayes regularization	Fewer parameters, better generalization	Extra gradient computation
HiddenKey	KL loss regularization	Combines dropout methods	Superior generalization	~50% longer training time
TriAdaptLoRA	Dynamic learning rates	Triangular splits, adaptive rank growth	Dynamic parameter allocation	Inefficient fixed thresholds
LSR-Adapt		Matrix LSR representations	Ultra-efficient parameter tuning	Underutilizes Kronecker products

IV. ARCHITECTURAL INNOVATION

The original LoRA framework revolutionized PEFT by achieving nearly full fine-tuning performance with reduced computational costs, yet faced constraints in multi-task adaptability and cross-domain generalization. Recent architectural innovations[43] have addressed these limitations through dynamic parameter allocation mechanisms, exemplified by the Mixture of Experts[44] with LoRA framework. This enhanced paradigm enables simultaneous optimization of specialized task modules and adaptive weight distribution through a learnable routing network, where each LoRA component functions as a domain-specific processor. The integrated training process not only preserves computational efficiency but also enhances model versatility through context-aware specialization, effectively overcoming previous bottlenecks in handling concurrent diverse objectives while maintaining minimal parameter overhead.

A. Dynamic combination method for multiple LoRA modules

The foundational exploration into dynamic adaptation strategies began with the introduction of Covariance Matrix Adaptation (CMA) in evolutionary algorithms. This pioneering work aimed to optimize parameter correlations by enabling the algorithm to learn and adapt the covariance matrix, thereby enhancing search efficiency in complex, high-dimensional spaces rather than directly addressing fixed mutation distributions. Subsequently, the emergence of Hydra was driven by the need to handle large-scale datasets and enable real-time analysis. Developed by AddThis to address their data processing challenges, Hydra[45] introduced a distributed task processing system capable of supporting both batch and stream processing, significantly enhancing data handling capabilities.

As the demand for efficient and scalable adaptation techniques grew, GLoRA[46] was introduced to expand applicability across diverse tasks. Recognizing the limitations of existing adapters in handling high-dimensional data and varying task requirements, GLoRA proposed a hierarchical search mechanism that generalized prompt modules, ensuring broader task compatibility and improved model flexibility. Building upon these advancements, LoraHub[47] further refined dynamic composition by leveraging principles inspired by CMA. Motivated by the desire to eliminate the need for extra parameters or gradients, LoraHub enabled on-the-fly combination of multiple LoRA modules, demonstrating remarkable few-shot performance through minimal example integration and automated module assembly.

LoRA-Flow[48] addressed the need for adaptive fusion mechanisms in generative tasks. By introducing dynamic fusion weights, LoRA-Flow optimized the influence of LoRA modules through learned parameters, ensuring optimal performance across varying generative scenarios and token-level adaptations. Meanwhile, the development of FLoRA[49] was driven by the requirement for efficient high-dimensional tensor adaptation. Recognizing the limitations of existing adapters in handling dense prediction tasks, FLoRA introduced a Tucker decomposition-based approach, significantly enhancing the efficiency and applicability of low-rank adaptation methods across diverse model architectures.

LoraRetriever[50] emerged to address the challenges of contextual adaptation in mixed-task scenarios. Inspired by the need for efficient retrieval and composition of LoRA modules, LoraRetriever enhanced contextual adaptation by leveraging input prompts to retrieve and compose relevant modules, supporting seamless transitions between tasks and improving overall system efficiency in dynamic environments. The key features and mechanisms of different structural components are shown in TABLE VI.

TABLE VI. COMPARISON OF DIFFERENT LORA COMPOSITION METHODS.

Variant	Key Features and Mechanism
Hydra	Multi-head adaptation, linear feature combination, no-inference delay
GLoRA	Generalized prompt module, hierarchical structure search, no extra inference cost
LoraHub	Dynamic LoRA composition, few-shot adaptation, no additional parameters/gradients
LoRA-Flow	Dynamic fusion weights via fusion gates, layer-wise fusion, generative task optimization
FLoRA	Batched low-rank adapters, throughput efficiency, flexible request handling
LoraRetriever	Input-aware retrieval, LoRA mixing/fusion strategies, batch inference optimization

B. Combination of Lora With Mixture of Experts

1) Evolution of MoE Architectures.

The foundational work by Shazeer et al.[43] introduced sparse-gated MoE layers, where a trainable gating network dynamically selects subsets of experts for each input token. This

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

mechanism used a softmax function over learned gating weights to assign tokens to experts, coupled with a load-balancing loss to prevent expert underutilization. Experiments on machine translation showed a 2048-expert MoE achieved 25% lower perplexity than dense models at equivalent computational costs, with experts specializing in syntactic patterns like article usage. Building on this, Switch Transformers[51] simplified routing to single-expert activation, reducing cross-device communication by 60% while enabling trillion-parameter training through expert-data parallelism. Their work demonstrated 7x faster pre-training for a 1.6T-parameter model compared to dense architectures. THOR[52] later enhanced robustness by injecting Gaussian noise into routing probabilities, mimicking neural dropout and improving adversarial task accuracy by 3.2%. SMoE-Dropout[53] further optimized generalization via curriculum learning: starting with 4 active experts and progressively reducing to 1, achieving 12% lower error on low-resource tasks.

2) Hybrid Architectures.

Zadouri et al.[54] pioneered lightweight MoE experts using LoRA, replacing dense layers with low-rank matrices to reduce parameters by 94% while retaining 97% accuracy. HydraLoRA[55] advanced this by decoupling task-specific features through asymmetric LoRA ranks—using rank 4 for input projections and rank 32 for output reconstructions—reducing multi-task interference by 41%. Mixture-of-LoRAs[56] addressed cross-task conflicts via explicit routing masks, isolating experts to predefined task clusters and cutting interference by 40%. MoLA[57] empirically demonstrated that deeper transformer layers benefit from more experts, proposing a pyramid allocation strategy that assigns 4 experts to early layers and 16 experts to later layers. This approach enhanced reasoning accuracy by 15% through concentrating adaptation capacity where semantic complexity reaches its peak.

3) Dynamic Routing Optimization.

MOE Meets LLMs[58] introduced temperature-scaled routing, where a meta-learned controller adjusts the softmax temperature inversely with task complexity—lower temperatures for deterministic expert selection in simple tasks, higher for exploration in complex ones. This reduced misassignments by 33% in cross-domain tasks. MoCLE[59] implemented a two-stage routing system: first clustering inputs into semantic groups via k-means, then routing clusters to specialized experts. This approach increased coverage of rare tasks from 58% to 92% with minimal latency overhead. For biomolecular modeling, X-LoRA[60] designed domain-aware gates that activate experts based on protein secondary structures, improving binding prediction accuracy by 8%. LLaVA-MoLE[61] tackled multimodal conflicts by allowing

TABLE VII. COMPARISON OF LORA WITH MIXTURE OF EXPERTS.

Variant	Basis	Scenarios	Key Features and Mechanism
Sparsely-Gated Mixture-of-Experts[43]	Sparse gating in a mixture-of-experts framework	Scaling neural networks	Dynamic sparse activation with a learned routing mechanism that selectively activates expert subnetworks
Switch Transformers[51]	Simplified sparse routing via a switch mechanism	Pre-training and fine-tuning across language tasks	Efficient and dynamic expert allocation with a streamlined routing design and improved computational efficiency
Stochastic Experts[52]	Stochastic activation with sparse gating	General transformer modulation in various tasks	Randomized expert selection to enhance sparsity and robustness while reducing over-specialization
SMoE-Dropout[53]	Integration of MoE with dropout-like regularization	Scaling dense transformer architectures	Combines sparse expert utilization with dropout-inspired mechanisms to achieve improved generalization and self-slimming capabilities
Extremely Parameter Efficient MoE[54]	Ultra-sparse expert routing	Instruction tuning	Emphasizes maximal parameter efficiency through extreme sparsity in expert allocation while maintaining competitive performance on tuning tasks
HydraLoRA[55]	Asymmetric LoRA architecture	Efficient and flexible fine-tuning	Introduces an asymmetric design that reduces parameter overhead while maintaining performance through diversified expert pathways
Mixture-of-LoRAs[56]	Collaborative low-rank adaptation	Multitask tuning	Leverages a mixture of low-rank components to facilitate efficient multi-task learning with shared parameter structures
Higher Layers Need More LoRA Experts[57]	Layer-specific LoRA augmentation	Enhancing deeper layers in large architectures	Dynamically allocates more LoRA experts in higher network layers, tailoring adaptation to the increased complexity of deep representations
MOE Meets LLMs[58]	Integration of MoE with LLM fine-tuning	Multi-task medical applications	Tailors MoE mechanisms for domain-specific tasks, focusing on multi-task adjustment and robust performance in specialized fields such as medicine
Cluster-conditional LoRA Experts[59]	Cluster-conditioned expert routing using LoRA modules	Vision-language instruction tuning	Implements clustering strategies to conditionally route inputs towards specialized LoRA experts for improved cross-modal performance
LLaVA-MoLE[61]	Sparse mixture of LoRA experts	Instruction finetuning amid data conflicts	Utilizes LoRA modules to mitigate data conflicts with a sparse expert mixture, enhancing adaptation during instruction finetuning
LoRAMoE[62]	Integration of LoRA with an MoE-style plugin	Mitigating world knowledge forgetting	Coupling LoRA adaptation with a MoE-style plugin to retain and update world knowledge efficiently during model updates
MoELoRA[63]	Contrastive learning guided MoE with LoRA integration	Parameter-efficient fine-tuning of large-scale models	Integrates contrastive guidance with LoRA-based MoE to balance and optimize expert contributions, enhancing retention and adaptation during finetuning
MoRAL[64]	MoE augmented LoRA framework	Lifelong learning for large language models	Augments traditional LoRA with MoE strategies to support continual adaptation and lifelong learning, effectively mitigating knowledge forgetting
OMoE[65]	Diversified mixture of low-rank adaptation via orthogonal finetuning	Enhancing diversity in adaptation across tasks	Employs orthogonal finetuning to diversify expert contributions and improve collaboration among low-rank adapted modules for broad, robust task performance

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

adjacent layers to share 50% of expert parameters, reducing alignment errors by 55% in visual QA tasks.

4) Catastrophic Forgetting Mitigation.

LoRAMoE[62] quantified knowledge retention using a Knowledge-Keeping Factor (KKF) , calculated as the historical activation frequency of each expert. When new tasks risked overwriting critical experts, pseudo-replay mechanisms reactivated them using stored task embeddings, limiting forgetting to 8% over 10 sequential tasks. MoELoRA[63] enforced expert specialization via contrastive loss, maximizing distance between expert outputs in embedding space, which reduced feature overlap by 52%. MoRAL[64] benchmarked lifelong learning on a 500-task dataset, showing dynamic expert expansion (adding 1 expert per 5 tasks) kept accuracy drops below 5% versus 25% in static MoE. OMoE[65] employed Stiefel manifold optimization to maintain expert orthogonality, preserving 98% diversity across 20 tasks by constraining weight matrices to remain perpendicular during training. TABLE I. Comparison of Lora With Mixture of Experts.

V. Conclusion

A. Challenges and discussion

Researchers have identified several issues when using LoRA for adapting large foundation models:

Theoretical Limitations: Although LoRA has proven efficient, its inner workings—especially how it preserves pretrained knowledge while modifying task-specific features—remain insufficiently explained. Existing theories often rely on simplified settings, leaving gaps for more complex scenarios.

Design Methodologies: Current implementations tend to follow empirical observations rather than established design principles. Critical questions remain open regarding where to place adapter modules, how to choose the appropriate low-rank dimensions across various layers, and whether non-Euclidean frameworks (e.g., hyperbolic spaces) might better capture model hierarchies.

Computational Overheads: As foundation models grow in scale, managing multiple concurrent LoRA adaptations becomes a bottleneck. Issues such as memory fragmentation and increased I/O cost—exacerbated by dynamic handling of adapter modules and key-value caches—present serious challenges.

Robustness, Verification, and Security: For deployment in sensitive applications such as healthcare or autonomous systems, it is crucial to ensure that LoRA-adapted models behave reliably, especially under adversarial conditions or distribution shifts. There is a strong need for rigorous uncertainty quantification and verification frameworks to safeguard against potential failures. In federated or cloud settings, protecting sensitive information from unauthorized access also remains an important concern.

B. Future prospects

Looking forward, the literature highlights several promising research directions to further exploit LoRA's advantages:

Modular Adaptation for Generative Services: In a Generative-as-a-Service (GaaS) scenario, one can envision the construction of distinct LoRA modules for individual functions. Such modularity would allow platforms to rapidly update services by combining or retraining modules—a key benefit given the fast pace of real-world changes.

Continued Domain-Specific Pre-Training: Adapting foundation models to specialized domains via ongoing self-supervised training is critical. However, the heavy computational load of this process calls for LoRA-based solutions that can reduce resource demands while maintaining performance.

Integration into Autonomous Agents: Future autonomous systems could benefit from replacing static prompts with LoRA-trained modules that assign and adapt roles based on large datasets. This approach would not only streamline the integration of specialized tools but also allow for the construction of more effective memory components—potentially overcoming current limitations associated with retrieval-augmented generation.

References

[1] A. Vaswani *et al.*, "Attention Is All You Need," Aug. 01, 2023, *arXiv*: arXiv:1706.03762. Accessed: Mar. 01, 2024. [Online]. Available: http://arxiv.org/abs/1706.03762

[2] OpenAI *et al.*, "GPT-4 Technical Report," Mar. 04, 2024, *arXiv*: arXiv:2303.08774. doi: 10.48550/arXiv.2303.08774.

[3] H. Touvron *et al.*, "LLaMA: Open and Efficient Foundation Language Models," Feb. 27, 2023, *arXiv*: arXiv:2302.13971. Accessed: May 15, 2024. [Online]. Available: http://arxiv.org/abs/2302.13971

[4] DeepSeek-AI *et al.*, "DeepSeek-V3 Technical Report," Dec. 27, 2024, *arXiv*: arXiv:2412.19437. doi: 10.48550/arXiv.2412.19437.

[5] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," Jan. 22, 2025, *arXiv*: arXiv:2501.12948. doi: 10.48550/arXiv.2501.12948.

[6] N. Ding *et al.*, "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nat Mach Intell*, vol. 5, no. 3, pp. 220–235, Mar. 2023, doi: 10.1038/s42256-023-00626-4.

[7] N. Houlsby *et al.*, "Parameter-Efficient Transfer Learning for NLP," Jun. 13, 2019, *arXiv*: arXiv:1902.00751. doi: 10.48550/arXiv.1902.00751.

[8] B. Lester, R. Al-Rfou, and N. Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning," Sep. 02, 2021, *arXiv*: arXiv:2104.08691. doi: 10.48550/arXiv.2104.08691.

[9] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," Oct. 16, 2021, *arXiv*: arXiv:2106.09685. Accessed: Sep. 27, 2024. [Online]. Available: http://arxiv.org/abs/2106.09685

[10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," May 23, 2023, *arXiv*: arXiv:2305.14314. Accessed: Oct. 08, 2024. [Online]. Available: http://arxiv.org/abs/2305.14314

[11] Q. Zhang *et al.*, "AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning," Dec. 20, 2023, *arXiv*: arXiv:2303.10512. Accessed: Sep. 18, 2024. [Online]. Available: http://arxiv.org/abs/2303.10512

[12] Y. Hao, Y. Cao, and L. Mou, "Flora: Low-Rank Adapters Are Secretly Gradient Compressors," Jun. 12, 2024, *arXiv*: arXiv:2402.03293. Accessed: Sep. 20, 2024. [Online]. Available: http://arxiv.org/abs/2402.03293

[13] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," Jul. 22, 2020, *arXiv*: arXiv:2005.14165. doi: 10.48550/arXiv.2005.14165.

[14] A. Aghajanyan, L. Zettlemoyer, and S. Gupta, "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning," Dec. 22, 2020, *arXiv*: arXiv:2012.13255. doi: 10.48550/arXiv.2012.13255.

[15] V. Fomenko, H. Yu, J. Lee, S. Hsieh, and W. Chen, "A Note on LoRA," Apr. 07, 2024, *arXiv*: arXiv:2404.05086. doi: 10.48550/arXiv.2404.05086.

[16] DeepSeek-AI *et al.*, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," Jun. 19, 2024, *arXiv*: arXiv:2405.04434. doi: 10.48550/arXiv.2405.04434.

[17] R. Qiang, R. Zhang, and P. Xie, "BiLoRA: A Bi-level Optimization Framework for Overfitting-Resilient Low-Rank Adaptation of Large Pre-

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

trained Models," Mar. 19, 2024, *arXiv*: arXiv:2403.13037. Accessed: Sep. 28, 2024. [Online]. Available: http://arxiv.org/abs/2403.13037

[18] M. Li, W. M. Si, M. Backes, Y. Zhang, and Y. Wang, "SaLoRA: Safety-Alignment Preserved Low-Rank Adaptation," Jan. 03, 2025, *arXiv*: arXiv:2501.01765. doi: 10.48550/arXiv.2501.01765.

[19] H. Nasiri and P. Garraghan, "EDoRA: Efficient Weight-Decomposed Low-Rank Adaptation via Singular Value Decomposition," Jan. 21, 2025, *arXiv*: arXiv:2501.12067. doi: 10.48550/arXiv.2501.12067.

[20] F. Zhang, L. Li, J. Chen, Z. Jiang, B. Wang, and Y. Qian, "IncreLoRA: Incremental Parameter Allocation Method for Parameter-Efficient Fine-tuning," Aug. 23, 2023, *arXiv*: arXiv:2308.12043. Accessed: Sep. 20, 2024. [Online]. Available: http://arxiv.org/abs/2308.12043

[21] N. Ding *et al.*, "Sparse Low-rank Adaptation of Pre-trained Language Models," Nov. 20, 2023, *arXiv*: arXiv:2311.11696. doi: 10.48550/arXiv.2311.11696.

[22] Z. Liu, J. Lyn, W. Zhu, X. Tian, and Y. Graham, "ALoRA: Allocating Low-Rank Adaptation for Fine-tuning Large Language Models," Apr. 15, 2024, *arXiv*: arXiv:2403.16187. doi: 10.48550/arXiv.2403.16187.

[23] S.-Y. Liu *et al.*, "DoRA: Weight-Decomposed Low-Rank Adaptation," Jul. 09, 2024, *arXiv*: arXiv:2402.09353. Accessed: Sep. 19, 2024. [Online]. Available: http://arxiv.org/abs/2402.09353

[24] R. Zhang, R. Qiang, S. A. Somayajula, and P. Xie, "AutoLoRA: Automatically Tuning Matrix Ranks in Low-Rank Adaptation Based on Meta Learning," Mar. 17, 2024, *arXiv*: arXiv:2403.09113. doi: 10.48550/arXiv.2403.09113.

[25] I. V. Oseledets, "Tensor-Train Decomposition," *SIAM J. Sci. Comput.*, vol. 33, no. 5, pp. 2295–2317, Jan. 2011, doi: 10.1137/090752286.

[26] Y. Yang, J. Zhou, N. Wong, and Z. Zhang, "LoRETTA: Low-Rank Economic Tensor-Train Adaptation for Ultra-Low-Parameter Fine-Tuning of Large Language Models," Feb. 18, 2024, *arXiv*: arXiv:2402.11417. doi: 10.48550/arXiv.2402.11417.

[27] A. Anjum, M. E. Eren, I. Boureima, B. Alexandrov, and M. Bhattarai, "Tensor Train Low-rank Approximation (TT-LoRA): Democratizing AI with Accelerated LLMs," Aug. 02, 2024, *arXiv*: arXiv:2408.01008. doi: 10.48550/arXiv.2408.01008.

[28] S. He, L. Ding, D. Dong, M. Zhang, and D. Tao, "SparseAdapter: An Easy Approach for Improving the Parameter-Efficiency of Adapters," Nov. 10, 2022, *arXiv*: arXiv:2210.04284. doi: 10.48550/arXiv.2210.04284.

[29] H. Zhou, X. Lu, W. Xu, C. Zhu, T. Zhao, and M. Yang, "LoRA-drop: Efficient LoRA Parameter Pruning based on Output Evaluation," Jun. 18, 2024, *arXiv*: arXiv:2402.07721. doi: 10.48550/arXiv.2402.07721.

[30] H. Wang, T. Liu, R. Li, M. Cheng, T. Zhao, and J. Gao, "RoseLoRA: Row and Column-wise Sparse Low-rank Adaptation of Pre-trained Language Model for Knowledge Editing and Fine-tuning," Oct. 15, 2024, *arXiv*: arXiv:2406.10777. doi: 10.48550/arXiv.2406.10777.

[31] M. Zhang *et al.*, "LoRAPrune: Structured Pruning Meets Low-Rank Parameter-Efficient Fine-Tuning," Aug. 07, 2024, *arXiv*: arXiv:2305.18403. doi: 10.48550/arXiv.2305.18403.

[32] Y. Hu, J. Zhang, X. Chen, Z. Zhao, C. Li, and H. Chen, "LoRS: Efficient Low-Rank Adaptation for Sparse Large Language Model," Jan. 15, 2025, *arXiv*: arXiv:2501.08582. doi: 10.48550/arXiv.2501.08582.

[33] Y. Li *et al.*, "LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models," Nov. 28, 2023, *arXiv*: arXiv:2310.08659. doi: 10.48550/arXiv.2310.08659.

[34] Y. Xu *et al.*, "QA-LoRA: Quantization-Aware Low-Rank Adaptation of Large Language Models," Oct. 09, 2023, *arXiv*: arXiv:2309.14717. doi: 10.48550/arXiv.2309.14717.

[35] C. Zhang, J. Cheng, G. A. Constantinides, and Y. Zhao, "LQER: Low-Rank Quantization Error Reconstruction for LLMs," May 30, 2024, *arXiv*: arXiv:2402.02446. doi: 10.48550/arXiv.2402.02446.

[36] H. Rajabzadeh *et al.*, "QDyLoRA: Quantized Dynamic Low-Rank Adaptation for Efficient Large Language Model Tuning," Feb. 16, 2024, *arXiv*: arXiv:2402.10462. doi: 10.48550/arXiv.2402.10462.

[37] D. Kalajdzievski, "A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA," Nov. 28, 2023, *arXiv*: arXiv:2312.03732. doi: 10.48550/arXiv.2312.03732.

[38] S. Hayou, N. Ghosh, and B. Yu, "LoRA+: Efficient Low Rank Adaptation of Large Models," Jul. 04, 2024, *arXiv*: arXiv:2402.12354. Accessed: Sep. 19, 2024. [Online]. Available: http://arxiv.org/abs/2402.12354

[39] Z. Lei, D. Qian, and W. Cheung, "Fast Randomized Low-Rank Adaptation of Pre-trained Language Models with PAC Regularization," in *Findings of the Association for Computational Linguistics ACL 2024*, Bangkok, Thailand and virtual meeting: Association for Computational Linguistics, 2024, pp. 5236–5249. doi: 10.18653/v1/2024.findings-acl.310.

[40] S. Wang, L. Chen, J. Jiang, B. Xue, L. Kong, and C. Wu, "LoRA Meets Dropout under a Unified Framework," May 27, 2024, *arXiv*: arXiv:2403.00812. doi: 10.48550/arXiv.2403.00812.

[41] Y. Liang, Y. Wang, and Y. Zeng, "TriAdaptLoRA: Brain-Inspired Triangular Adaptive Low-Rank Adaptation for Parameter-Efficient Fine-Tuning," Jan. 14, 2025, *arXiv*: arXiv:2501.08008. doi: 10.48550/arXiv.2501.08008.

[42] X. Li and A. Sarwate, "LSR-Adapt: Ultra-Efficient Parameter Tuning with Matrix Low Separation Rank Kernel Adaptation," Feb. 19, 2025, *arXiv*: arXiv:2502.13568. doi: 10.48550/arXiv.2502.13568.

[43] N. Shazeer *et al.*, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," Jan. 23, 2017, *arXiv*: arXiv:1701.06538. doi: 10.48550/arXiv.1701.06538.

[44] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, "A Survey on Mixture of Experts in Large Language Models," Apr. 09, 2025. doi: 10.1109/TKDE.2025.3554028.

[45] S. Kim, H. Yang, Y. Kim, Y. Hong, and E. Park, "Hydra: Multi-head Low-rank Adaptation for Parameter Efficient Fine-tuning," Sep. 13, 2023, *arXiv*: arXiv:2309.06922. doi: 10.48550/arXiv.2309.06922.

[46] A. Chavan, Z. Liu, D. Gupta, E. Xing, and Z. Shen, "One-for-All: Generalized LoRA for Parameter-Efficient Fine-tuning," Oct. 16, 2023, *arXiv*: arXiv:2306.07967. doi: 10.48550/arXiv.2306.07967.

[47] C. Huang, Q. Liu, B. Y. Lin, T. Pang, C. Du, and M. Lin, "LoraHub: Efficient Cross-Task Generalization via Dynamic LoRA Composition," Aug. 19, 2024, *arXiv*: arXiv:2307.13269. doi: 10.48550/arXiv.2307.13269.

[48] H. Wang *et al.*, "LoRA-Flow: Dynamic LoRA Fusion for Large Language Models in Generative Tasks," Feb. 18, 2024, *arXiv*: arXiv:2402.11455. doi: 10.48550/arXiv.2402.11455.

[49] Y. Wen and S. Chaudhuri, "Batched Low-Rank Adaptation of Foundation Models," Apr. 25, 2024, *arXiv*: arXiv:2312.05677. doi: 10.48550/arXiv.2312.05677.

[50] Z. Zhao *et al.*, "LoraRetriever: Input-Aware LoRA Retrieval and Composition for Mixed Tasks in the Wild," Feb. 15, 2024, *arXiv*: arXiv:2402.09997. doi: 10.48550/arXiv.2402.09997.

[51] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," Jun. 16, 2022, *arXiv*: arXiv:2101.03961. doi: 10.48550/arXiv.2101.03961.

[52] S. Zuo *et al.*, "Taming Sparsely Activated Transformer with Stochastic Experts," Feb. 03, 2022, *arXiv*: arXiv:2110.04260. doi: 10.48550/arXiv.2110.04260.

[53] T. Chen, Z. Zhang, A. Jaiswal, S. Liu, and Z. Wang, "Sparse MoE as the New Dropout: Scaling Dense and Self-Slimmable Transformers," Mar. 02, 2023, *arXiv*: arXiv:2303.01610. doi: 10.48550/arXiv.2303.01610.

[54] T. Zadouri, A. Üstün, A. Ahmadian, B. Ermiş, A. Locatelli, and S. Hooker, "Pushing Mixture of Experts to the Limit: Extremely Parameter Efficient MoE for Instruction Tuning," Sep. 11, 2023, *arXiv*: arXiv:2309.05444. doi: 10.48550/arXiv.2309.05444.

[55] C. Tian, Z. Shi, Z. Guo, L. Li, and C. Xu, "HydraLoRA: An Asymmetric LoRA Architecture for Efficient Fine-Tuning," May 23, 2024, *arXiv*: arXiv:2404.19245. doi: 10.48550/arXiv.2404.19245.

[56] W. Feng, C. Hao, Y. Zhang, Y. Han, and H. Wang, "Mixture-of-LoRAs: An Efficient Multitask Tuning for Large Language Models," Mar. 06, 2024, *arXiv*: arXiv:2403.03432. doi: 10.48550/arXiv.2403.03432.

[57] C. Gao *et al.*, "Higher Layers Need More LoRA Experts," Feb. 13, 2024, *arXiv*: arXiv:2402.08562. doi: 10.48550/arXiv.2402.08562.

[58] Q. Liu *et al.*, "When MOE Meets LLMs: Parameter Efficient Fine-tuning for Multi-task Medical Applications," May 31, 2024, *arXiv*: arXiv:2310.18339. doi: 10.48550/arXiv.2310.18339.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.

[59] Y. Gou *et al.*, "Mixture of Cluster-conditional LoRA Experts for Vision-language Instruction Tuning," Jul. 04, 2024, *arXiv*: arXiv:2312.12379. doi: 10.48550/arXiv.2312.12379.

[60] E. L. Buehler and M. J. Buehler, "X-LoRA: Mixture of Low-Rank Adapter Experts, a Flexible Framework for Large Language Models with Applications in Protein Mechanics and Molecular Design," Mar. 30, 2024, *arXiv*: arXiv:2402.07148. doi: 10.48550/arXiv.2402.07148.

[61] S. Chen, Z. Jie, and L. Ma, "LLaVA-MoLE: Sparse Mixture of LoRA Experts for Mitigating Data Conflicts in Instruction Finetuning MLLMs," Jan. 30, 2024, *arXiv*: arXiv:2401.16160. doi: 10.48550/arXiv.2401.16160.

[62] S. Dou *et al.*, "LoRAMoE: Alleviating World Knowledge Forgetting in Large Language Models via MoE-Style Plugin," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 1932–1945. doi: 10.18653/v1/2024.acl-long.106.

[63] T. Luo *et al.*, "MoELoRA: Contrastive Learning Guided Mixture of Experts on Parameter-Efficient Fine-Tuning for Large Language Models," Feb. 20, 2024, *arXiv*: arXiv:2402.12851. doi: 10.48550/arXiv.2402.12851.

[64] S. Yang, M. A. Ali, C.-L. Wang, L. Hu, and D. Wang, "MoRAL: MoE Augmented LoRA for LLMs' Lifelong Learning," Feb. 17, 2024, *arXiv*: arXiv:2402.11260. doi: 10.48550/arXiv.2402.11260.

[65] J. Feng, Z. Pu, T. Hu, D. Li, X. Ai, and H. Wang, "OMoE: Diversifying Mixture of Low-Rank Adaptation by Orthogonal Finetuning," Jan. 17, 2025, *arXiv*: arXiv:2501.10062. doi: 10.48550/arXiv.2501.10062.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:33:28 UTC from IEEE Xplore. Restrictions apply.